KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Biomedical Engineering

Course No: BME 4232

Project Title: MediConnect – A Video Conferencing App for Telemedicine and Healthcare

Submitted to

Anik Ghosh

Lecturer

Department of Biomedical Engineering

Khulna University of Engineering & Technology (KUET)

Submitted to

Amit Dutta Roy

Lecturer

Department of Biomedical Engineering

Khulna University of Engineering & Technology (KUET)

Submitted by:

Sourav Basak Shuvo

Roll No. 1815010

4th year, 2nd Semester

Department of Biomedical Engineering

Khulna University of Engineering & Technology (KUET)

Date of submission: 04-12-2023

Index

LIST OF FIGURES

Abstract

The use of technology into healthcare has fundamentally transformed the manner in which medical services are provided, notably via the utilization of telemedicine. The focus of this project is to create an advanced Video Conferencing App designed exclusively for telemedicine and healthcare services. The app is developed using Android Studio and utilizes the Java language. It smoothly incorporates Firebase and ZEGOCLOUD, providing a strong framework for remote medical consultations. The Java programming language serves as the fundamental framework for the application's functioning. Android Studio functioned as the principal Integrated Development Environment (IDE), facilitating agile and effective Android application development. The outcome is a groundbreaking and easy-to-use technology that has the potential to transform healthcare accessibility and patient care by enabling secure and top-notch remote consultations. This platform aims to transform the provision of healthcare services by giving utmost importance to data security, user-friendliness, and the integration of electronic health records. It seeks to overcome geographical limitations and guarantee accessible, top-notch medical treatment for everyone.

CHAPTER 1

Introduction

1.1 Telehealth and Telemedicine

Telehealth and telemedicine are frequently used interchangeably, however telehealth has expanded to encompass a wider range of digital healthcare activities and services. In order to comprehend the juxtaposition of telehealth and telemedicine, it is imperative to initially establish a precise definition of telemedicine.

Various technologies are being utilized for telehealth, such as mHealth (or mobile health), video and audio technology, digital photography, remote patient monitoring (RPM), and store and forward technologies [1].

1.2 Teleconferencing

A teleconference, often known as a telecon, is a real-time exchange of information between multiple individuals who are physically separated but connected by a telecommunications system. Teleconferencing is a term that is frequently interchangeably used with audio conferencing, telephone conferencing, and phone conferencing. The telecommunications system can facilitate teleconferencing by offering audio, video, and/or data services through many modes, including telephone, computer, telegraph, teletypewriter, radio, and television [2].

1.3 Video conferencing

Video conferencing is a real-time, visual communication method that enables many remote participants to interact over the internet, replicating an in-person conference. Video conferencing is crucial as it facilitates the connection between those who would typically be unable to establish a direct, in-person interaction.

Video conferencing, in its most basic form, enables the transmission of still images and written content between two different places. At its highest level of complexity, it enables the transfer of moving images and superior audio between numerous sites. Video conferencing relies on VoIP, the technology that enables voice communication over the internet. VoIP utilizes codecs, which are specialized algorithms, to facilitate the transmission of audio and video signals

between two sites. The video conferencing process can be divided into two distinct stages: compression and transfer [3].

During the process of compression, the camera and microphone record analogue audiovisual (AV) input. The collected data manifests as a series of uninterrupted waves characterized by their frequencies and amplitudes. To transmit this data over a standard network, it is necessary to employ codecs to compress the data into digital packets. During the transfer phase, the compressed data is transmitted via the digital network to the recipient computer. Upon reaching the endpoint, the codecs proceed to decompress the data. The codecs perform the task of converting the digital audio and video signals back into their original analogue form. This allows the display screen and audio speakers to accurately perceive and reproduce the audiovisual information [4].

Integrated web cams in laptop, tablet, or desktop computers serve the purpose of facilitating video conferencing. Video conferences can be joined using smartphones equipped with cameras. The primary characteristics of video conferencing are screen sharing, simultaneous annotation, chat box, file sharing, video call recording, noise cancellation, and device switching. Video conferencing is especially beneficial for distant workers, who are likely to depend on these services to host the majority of their work-related meetings. Telemonitoring and remote learning are commonly linked to video conferencing in contemporary technologies [5].

Figure 1.1 Video Conferencing

1.4 Video Conferencing in Telemedicine

Currently, telemedicine video conferencing enables you to have face-to-face interactions with your patients regardless of their location, while also allowing you to observe their nonverbal clues throughout conversation. The patient can ambulate within their living room to demonstrate their gait or utilize their digital device to exhibit the progress of their wound healing. They can explain their situation in the same manner as if they had visited your office. Consequently, you will be able to interact with your patients in a more effective and reliable way. Modern browser-based video conferencing solutions are easily accessible and user-friendly on many devices from any location, as long as there is a dependable internet connection. WebRTC standards allow for the inclusion of any digital device in video conferencing, while also facilitating secure browser-based telemedicine. The capabilities of telemedicine video conferencing technology efficiently erase geographical limitations that have long hindered patients from getting high-quality medical care. Irrespective of their geographical location, you can establish direct and instantaneous communication with your patients by utilizing telemedicine over a video conference call [6].

Chapter 2

Background Study

2.1 History of Video Conferencing

The concept of video communication and video conferencing, similar to numerous other technologies, was much ahead of its era. In the late 1800s, people expressed discontent with the limited experience of solely auditory communication through the telephone and desired the ability to visually perceive the individuals they were conversing with.

Starting from the initial video talk and progressing via other stops, including dozen-person Zoom conversations, the journey was lengthy. AT&T and Bell Labs, being the sole telecommunications providers for an extended period, were the trailblazers in the advancement of video call technology.

Video conferencing has seen substantial evolution, progressing from one-on-one transmissions using black and white still images to multi-party transmissions in real-time and 4K quality. We own an extensive and distinguished past that encompasses Zoom, Teams, Skype, and Webex.

Webex is the most ancient among the group. Established in 1995, it was subsequently acquired by Cisco Systems in 2007. Cisco introduced a wide range of products for corporate users. However, in September 2020, they unveiled a new platform called Webex Classrooms, specifically designed for virtual homerooms.

Skype was launched in August 2003 by three Estonian software developers. eBay acquired Skype in 2005 and subsequently sold it to Microsoft in 2011. Initially, it functioned solely as a text-based messaging platform, but subsequently incorporated video functionality.

Microsoft Teams made its debut in 2017, arriving slightly behind schedule. It was intended for Microsoft Classroom and Skype for Business to be replaced by it. The features encompassed are individual and group chat, file sharing, team chats, team channels, and Voice over Internet Protocol (VoIP). Furthermore, it provides real-time transcriptions of audio conversations in written format.

Zoom Cloud Meetings, also referred to as Zoom, is an alternative option for conducting video conferences. Zoom allows for large-scale conference calls with a maximum capacity of 100

people, and up to 49 individuals can be simultaneously displayed on a desktop monitor.

2.2 Motivation:

The necessity for safe, effective, and easily accessible healthcare delivery is the driving force behind the development of a telemedicine video conferencing app. There is a growing need for remote medical consultations and interventions as we traverse an era of unparalleled challenges. Geographical limitations, the worldwide pandemic, and restricted access to healthcare facilities are just a few of the factors that have highlighted how urgent it is to find creative solutions that go beyond conventional healthcare bounds.

Our goal is to close these gaps by utilizing technology to enable smooth communication between patients and healthcare professionals. The proposed video conferencing app is more than just a tool; it's a ray of hope that will enable people to get prompt medical advice, diagnosis, and treatment from the convenience of their own homes.

Furthermore, this program reflects our dedication to using innovation to advance society. By cultivating a platform that facilitates healthcare accessibility, we are in line with the overarching goal of promoting healthcare fairness and inclusivity, guaranteeing that every person, irrespective of location or situation, may obtain high-quality medical treatment.

In addition to outlining the technical requirements for creating such an app, our goal in producing this study is to demonstrate its revolutionary potential for completely changing the healthcare industry. The process of developing this video conferencing app is a reflection of our commitment to enhancing healthcare delivery and significantly impacting people's lives all around the world, not just a technical undertaking.

2.3 Objectives:

The main objectives of this project are to

- Develop video conferencing application.
- Securely connect people over distance through the app.
- Build up a system for video calling and data transfer.
- Be able to provide emergency and real-time telemedicine service.

Chapter 3

Methodology

3.1 Main Flow Diagram

At first, the interface of the app is designed using Android Studio. Then**,** Other pages such as, Sign Up, Sign In, Forgot Password pages are built and connected. Firebase will be integrated for Sign up, Sign In page data authentication and other data storing. In the meeting page, “Create a Meeting”, “Join a Meeting” section and “Call”, “Message” and “Logout” Button are created. Here, Zegocould was integrated and used for teleconferencing, chatting & image sharing. After building the app, it is tested for error and bugs are fixed.

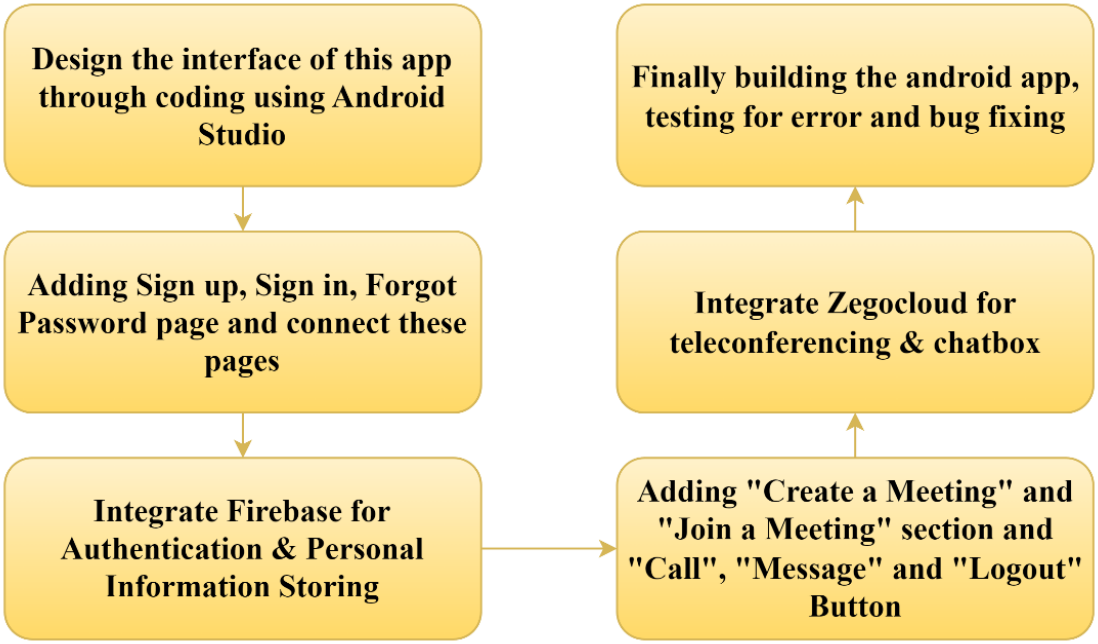


Figure 3.1 Flowchart of the Designed Application

3.2 Major Software Components

3.2.1 Android Studio

Android Studio serves as the designated Integrated Development Environment (IDE) for the purpose of Android application development. Android Studio provides additional functionalities that improve efficiency in developing Android applications, leveraging the robust code editor and developer tools of IntelliJ IDEA.

- A versatile build system based on Gradle

- An emulator that is both speedy and packed with advanced functionalities.
- An integrated platform that enables development for all Android devices
- Enable real-time updates of composables on both emulators and physical devices with Live Edit.
- Utilize code templates and integrate with GitHub to facilitate the development of standard application features and import pre-existing code examples.
- Comprehensive testing tools and frameworks
- Lint tools are used to detect and identify issues related to performance, usability, version compatibility, and other potential difficulties.
- Support for C++ and the NDK
- Includes native compatibility for Google Cloud Platform, simplifying the process of incorporating Google Cloud Messaging and App Engine [7].

3.2.2 Firebase

The Firebase Realtime Database is a database that is hosted in the cloud. The data is kept in the form of JSON and is continuously synchronized with all connected clients in real-time. By utilizing our Apple iOS, Android, and JavaScript SDKs, you can construct cross-platform applications that enable all of your clients to access a single Realtime Database instance. This ensures that they will automatically receive updates including the most recent data.

Alternatively, you may want to explore using Cloud Firestore for contemporary applications that demand more complex data structures, the capacity to do queries, the ability to scale, and increased availability.

The Firebase Realtime Database enables the development of sophisticated, collaborative apps by granting secure access to the database straight from the client-side code. The data is stored locally and is accessible even when there is no internet connection. Additionally, real-time events are still triggered, ensuring that the end user has a prompt and interactive experience. Upon reestablishing connectivity, the Realtime Database seamlessly integrates the local data modifications with the remote updates that took place during the client's offline period, instantly resolving any conflicts that may arise.

The Realtime Database offers Firebase Realtime Database Security Rules, a versatile rules language based on expressions, which allows you to specify the structure of your data and

determine when data can be accessed or modified. By integrating with Firebase Authentication, developers may establish the authorization rules for data access, specifying which users are granted access and the specific permissions they have.

The Realtime Database is a type of database known as NoSQL, which means it operates differently and offers distinct optimizations and capabilities when compared to a relational database. The Realtime Database API is specifically designed to exclusively permit actions that can be swiftly completed. By utilizing this feature, you may construct an exceptional real-time encounter that is capable of accommodating a vast number of customers while maintaining optimal responsiveness. Hence, it is crucial to consider the manner in which users require access to your data and subsequently organize it in a suitable manner [8].

3.2.3 ZEGOCLOUD

ZEGOCLOUD, established in 2015, is a worldwide provider of cloud communication services. It empowers organizations and developers to effortlessly and expeditiously acquire real-time audio and video communication capabilities by integrating a unified Software Development Kit (SDK).

ZEGOCLOUD has successfully delivered top-notch services to consumers across over 200 countries and regions. As a result, it has earned the trust and satisfaction of several prominent customers in various industries such as social media, gaming, live streaming, finance, education, medical, and intelligent hardware [9].

Chapter 4

Implementation

4.1 Code Implementation:

This App was build using Android studio, java language, Firebase and ZEGOCLOUD. The codes were written in java language and Android Studio was used as Integrated Development Environment (IDE) for the purpose of Android application development. The major codes for this project are discussed in this section

Code for "main.dart":

```
import 'package:TeleCon/screens/signin_screen.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:zego_zimkit/zego_zimkit.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  ZIMAppConfig appConfig = ZIMAppConfig();
  appConfig.appID = 1261518879;
  appConfig.appSign =
'76a8d22d0d2faff6f24c5a8c88ce8b74e3b2653e7ad704463bc7005ab313dbe3';

  ZIM.create(appConfig);
  ZIMKit().init(
    appID: 1261518879, // your appid
    appSign:

'76a8d22d0d2faff6f24c5a8c88ce8b74e3b2653e7ad704463bc7005ab313dbe3',
// your appSign
  );
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor:
Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const SignInScreen(),
    );
  }
}
```

The provided code in the "main.dart" file does the following tasks: it initializes Firebase, configures the Zego ZIMKit SDK, defines the root widget of the Flutter app as "MyApp", and sets the initial screen to a “SignInScreen”.

Code for “call.dart”:

```
import 'package:flutter/cupertino.dart';
import 'package:zego_uikit_prebuilt_call/zego_uikit_prebuilt_call.dart';
import 'const.dart';

class MyCall extends StatelessWidget {
  const MyCall({Key? key, required this.callID,required
this.UserId,required this.UserName}) : super(key: key);
  final String callID;
  final String UserId;
  final String UserName;

  @override
  Widget build(BuildContext context) {
    return ZegoUIKitPrebuiltCall(
```

```
      appID: MyConst
          .appId, // Fill in the appID that you get from ZEGOCLOUD
Admin Console.
      appSign: MyConst
          .appSign, // Fill in the appSign that you get from
ZEGOCLOUD Admin Console.
      userID: UserId,
      userName: UserName,
      callID: callID,
      // You can also use groupVideo/groupVoice/oneOnOneVoice to
make more types of calls.
      config: ZegoUIKitPrebuiltCallConfig.oneOnOneVideoCall()
        ..onOnlySelfInRoom = (context) =>
Navigator.of(context).pop(),
    );
  }
}
```

The call.dart file contains code that specifies a "MyCall" class in Flutter. This class is responsible for constructing a user interface layout that utilises the Zego UIKit Prebuilt Call module to enable video calling functionality. The function requires crucial inputs, including callID, UserId, and UserName, to initiate a call session. The build method use the "ZegoUIKitPrebuiltCall" widget to configure the appID and appSign acquired from ZEGOCLOUD Admin Console. It also sets up the user's ID, name, and call information. In addition, it delineates a video call setup for two participants and establishes a procedure to immediately end the connection when the user is the only remaining participant in the room, using the "onOnlySelfInRoom" callback.

Code for “signin_screen.dart”:

```
import 'package:TeleCon/screens/reset_password.dart';
import 'package:TeleCon/screens/signup_screen.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';

import '../call.dart';
import '../linkscreen.dart';
```

```dart
import '../reusable_widgets/reusable_widget.dart';
import '../utils/color_utils.dart';


class SignInScreen extends StatefulWidget {
  const SignInScreen({Key? key}) : super(key: key);

  @override
  _SignInScreenState createState() => _SignInScreenState();
}

class _SignInScreenState extends State<SignInScreen> {
  TextEditingController _passwordTextController =
TextEditingController();
  TextEditingController _emailTextController =
TextEditingController();
  late String errorCode;
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        width: MediaQuery.of(context).size.width,
        height: MediaQuery.of(context).size.height,
        decoration: BoxDecoration(
            gradient: LinearGradient(colors: [
              hexStringToColor("CB2B93"),
              hexStringToColor("9546C4"),
              hexStringToColor("5E61F4")
            ], begin: Alignment.topCenter, end:
Alignment.bottomCenter)),
        child: SingleChildScrollView(
          child: Padding(
            padding: EdgeInsets.fromLTRB(
                20, MediaQuery.of(context).size.height * 0.2, 20,
0),
            child: Column(
              children: <Widget>[
```

```
                Image(image: AssetImage('assets/images/logo1.png')),
                const SizedBox(
                  height: 30,
                ),
                reusableTextField("Enter UserName",
Icons.person_outline, false,
                    _emailTextController),
                const SizedBox(
                  height: 20,
                ),
                reusableTextField("Enter Password",
Icons.lock_outline, true,
                    _passwordTextController),
                const SizedBox(
                  height: 5,
                ),
                forgetPassword(context),
                firebaseUIButton(context, "Sign In", () {
                  FirebaseAuth.instance
                      .signInWithEmailAndPassword(
                      email: _emailTextController.text,
                      password: _passwordTextController.text)
                      .then((value) {
                    Navigator.push(context,
                        MaterialPageRoute(builder: (context) =>
LinkScreen()));
                    //MyCall(callID: "1")
                  }).onError((error, stackTrace) {
                    print("Error ${error.toString()}");
                    Fluttertoast.showToast(
                      msg: "Invalid Login Credentials",
                      toastLength: Toast.LENGTH_SHORT,
                      gravity: ToastGravity.BOTTOM,
                      backgroundColor: Colors.grey,
                      textColor: Colors.white,
                      fontSize: 16.0,
                    );
```

```
                  });
                }),
                signUpOption()
              ],
            ),
          ),
        ),
      ),
    );
  }
  Row signUpOption() {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Text("Don't have account?",
            style: TextStyle(color: Colors.white70)),
        GestureDetector(
          onTap: () {
            Navigator.push(context,
                MaterialPageRoute(builder: (context) =>
SignUpScreen()));
          },
          child: const Text(
            " Sign Up",
            style: TextStyle(color: Colors.white, fontWeight:
FontWeight.bold),
          ),
        )
      ],
    );
  }

  Widget forgetPassword(BuildContext context) {
    return Container(
      width: MediaQuery.of(context).size.width,
      height: 35,
      alignment: Alignment.bottomRight,
```

```
      child: TextButton(
        child: const Text(
          "Forgot Password?",
          style: TextStyle(color: Colors.white70),
          textAlign: TextAlign.right,
        ),
        onPressed: () => Navigator.push(
            context, MaterialPageRoute(builder: (context) =>
ResetPassword())),
      ),
    );
  }
}
```

The code in signin_screen.dart establishes a Flutter widget called "SignInScreen" that handles user authentication in the application. The user interface showcases text boxes where users may input their email and password credentials. Additionally, there is a "Sign In" button that allows users to identify themselves with Firebase Authentication. The interface also provides choices for travelling to the sign-up page or resetting the password. The user interface has a gradient backdrop, input fields for both username and password, a "Forgot Password?" text button that directs users to a password reset page, and a "Sign Up" option specifically designed for new users. After a successful login, the user is sent to the "LinkScreen" where they may use meeting and chat features. If there is a problem in the authentication process, a toast message is sent to inform the user that their login credentials are incorrect.

Code for “linkscreen.dart”:

```
import 'dart:math';
import 'package:TeleCon/screens/signin_screen.dart';
import 'package:TeleCon/utils/color_utils.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
import 'package:zego_zimkit/zego_zimkit.dart';
import 'ChatUI.dart';
import 'call.dart';
```

```
String generateRandomStringAndNumber() {
  String randomString = generateRandomString(4);
  int randomNumber = generateRandomNumber(1000);
  String result = '$randomString$randomNumber';
  return result;
}

String generateRandomString(int length) {
  const chars = 'abcdefghijklmnopqrstuvwxyz0123456789';
  Random random = Random();
  return String.fromCharCodes(Iterable.generate(
    length,
    (_) => chars.codeUnitAt(random.nextInt(chars.length)),
  ));
}

int generateRandomNumber(int max) {
  Random random = Random();
  return random.nextInt(max);
}

class LinkScreen extends StatefulWidget {
  const LinkScreen({super.key});

  @override
  State<LinkScreen> createState() => _LinkScreenState();
}

class _LinkScreenState extends State<LinkScreen> {
  TextEditingController _idController = TextEditingController();
  TextEditingController _nameController = TextEditingController();
  TextEditingController _createmeeting= TextEditingController();
  TextEditingController _joinmeeting= TextEditingController();
  TextEditingController _hintcontroller= TextEditingController();
  bool showTextField = false;
  bool showjoinmeeting = false;
```

```
  User? user = FirebaseAuth.instance.currentUser;
  var docid;
  var username=FirebaseAuth.instance.currentUser?.displayName;
  var email = FirebaseAuth.instance.currentUser?.email;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Chat"),
        backgroundColor: hexStringToColor("CB2B93"),
        actions: <Widget>[

          ElevatedButton.icon(onPressed: () async {await
ZIMKit().connectUser(id: email!, name:
username!);Navigator.push(context,
              MaterialPageRoute(builder: (context) => ChatUI(email:
email, username: username,)));}, icon: Icon(Icons.message), label:
Text("Message"),),
        ],
      ),
      body: SingleChildScrollView(
        child: Container(
          width: MediaQuery.of(context).size.width,
          height: MediaQuery.of(context).size.height,
          decoration: BoxDecoration(
              gradient: LinearGradient(colors: [
            hexStringToColor("CB2B93"),
            hexStringToColor("9546C4"),
            hexStringToColor("5E61F4")
          ], begin: Alignment.topCenter, end:
Alignment.bottomCenter)),
          child: Column(
            children: [

              StreamBuilder(
                stream:
```

```
FirebaseFirestore.instance.collection('users').where('email',
isEqualTo: user?.email).snapshots(),
                builder: (context, AsyncSnapshot<QuerySnapshot>
snapshot) {
                  if (snapshot.connectionState ==
ConnectionState.waiting) {
                  return Center(child: CircularProgressIndicator());
                  }
                  else if (snapshot.hasError) {
                  return Center(child: Text('Error:
${snapshot.error}'));
                  }
                  else {
                    DocumentSnapshot document =
snapshot.data!.docs.first;
                    Map<String, dynamic> data = document.data() as
Map<String, dynamic>;
                    docid = document.id;
                    username = data['name'];
                    return SingleChildScrollView(
                      child: Padding(
                        padding: EdgeInsets.fromLTRB(
                            20, MediaQuery
                            .of(context)
                            .size
                            .height * 0.1, 20, 0),
                        child: Column(
                          children: <Widget>[
                        Container(

                          margin: EdgeInsets.symmetric(vertical: 8,
horizontal: 0),
                          decoration: BoxDecoration(
                            color: Colors.white,
                            borderRadius: BorderRadius.circular(20),
                            boxShadow: [
                              BoxShadow(
```

```
                                color: Colors.grey.withOpacity(0.5),
                                spreadRadius: 2,
                                blurRadius: 15,
                                offset: Offset(0, 3),
                              ),
                            ],
                          ),

                          child: ListTile(

                            title: Text(
                              data['name'],
                              style: TextStyle(
                                fontWeight: FontWeight.bold,
                                fontSize: 28,
                              ),
                              textAlign: TextAlign.center,
                            ),
                            subtitle: Text(
                              data['email'],
                              textAlign: TextAlign.center,
                            ),
                          ),
                        ),
                      ),
                          SizedBox(
                            height: 60,
                          ),
                          ElevatedButton(
                            onPressed: () {
                              setState(() {
                                if (!showTextField) {
                                  showTextField = true;
                                  _createmeeting.text =

generateRandomStringAndNumber();
                                }
                                else {
```

```
                                    showTextField = false;
                                  }
                                });
                              },
                              child: Text('Create a New Meeting'),
                            ),
                            if (showTextField)
                              Center(
                                child: Padding(
                                  padding: EdgeInsets.all(16.0),
                                  child: Row(
                                    mainAxisAlignment:
MainAxisAlignment.center,
                                    children: [
                                      SizedBox(
                                        width: 200,
                                        child: TextField(
                                          controller:
_createmeeting,
                                          textAlign:
TextAlign.center,
                                          style: TextStyle(
                                            fontSize: 24,
                                            fontWeight:
FontWeight.bold,
                                          ),
                                          readOnly: true,
                                          decoration:
InputDecoration(),
                                        ),
                                      ),
                                      ElevatedButton.icon(
                                        onPressed: ()
{Navigator.push(
                                          context,
                                          MaterialPageRoute(
                                            builder: (context) =>
```

```
                                                MyCall(
                                                  callID:
_createmeeting.text,
                                                  UserId:
document.id,
                                                  UserName:
data['name'],
                                                ),
                                              ),
                                            );},
                                            icon:
Icon(Icons.call_outlined),
                                            label: Text(""),
                                          ),
                                        ],
                                      ),
                                    ),
                                  ),


                              SizedBox(
                                height: 20,
                              ),
                              Text("Or,", textAlign: TextAlign.center,
                                style: TextStyle(fontSize: 24,
fontWeight: FontWeight
                                    .bold, color: Colors.white70),),

                              SizedBox(
                                height: 20,
                              ),
                              ElevatedButton(
                                onPressed: () {
                                  setState(() {
                                    if (!showjoinmeeting) {
                                      showjoinmeeting = true;
```

```
                        _joinmeeting.text = "";

                      }
                      else {
                        showjoinmeeting = false;
                      }
                    });
                  },
                  child: Text('Join a Meeting'),
                ),

              ],
            ),
          ),
        );
      }
    }
  ),
  if (showjoinmeeting)
    Center(
      child: Column(
        children: [
          Padding(
            padding: EdgeInsets.all(16.0),
            child: SizedBox(
              width: 200,
              child: TextField(
                controller: _joinmeeting,
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 24,
                  fontWeight: FontWeight.bold,
                ),
                decoration: InputDecoration(
                  hintText: "Enter the link",
                ),
              ),
```

```
                    ),
                  ),
                  ElevatedButton(
                    onPressed: () async {
                       if (_joinmeeting.text != null &&
                          _joinmeeting.text.isNotEmpty) {

                         Navigator.push(
                           context,
                           MaterialPageRoute(
                             builder: (context) =>
                                 MyCall(
                                   callID: _joinmeeting.text,
                                   UserId: docid,
                                   UserName: username!,
                                 ),
                           ),
                         );
                       }
                       else
                       {
                         Fluttertoast.showToast(
                           msg: "Please enter the link",
                           toastLength: Toast.LENGTH_SHORT, //
Duration for the toast message
                           gravity: ToastGravity.BOTTOM, //
Position of the toast message on the screen
                           backgroundColor: Colors.grey,
                           textColor: Colors.white,
                           fontSize: 16.0,
                         );
                       }
                    },
                    child: Icon(Icons.call_outlined),
                  )
                ],
              ),
```

```
                ),
              if (showjoinmeeting)
                const SizedBox(
                  height: 110,
                ),
              if (!showjoinmeeting)
                const SizedBox(
                height: 250,
                ),

              SizedBox(
                width: 300,
                child: InkWell(
                  onTap: () {
                    FirebaseAuth.instance.signOut().then((value) {
                      Navigator.push(
                          context,
                          MaterialPageRoute(
                              builder: (context) =>
SignInScreen()));
                    });
                  },
                  child: Container(
                    padding: EdgeInsets.all(12),
                    decoration: BoxDecoration(
                      color: Colors.transparent,
                      borderRadius: BorderRadius.circular(20),
                      border: Border.all(
                          color: Colors
                              .white), // Optional: Add a border
                    ),
                    child: Center(
                      child: Text(
                        "Logout",
                        style: TextStyle(color: Colors.white),
                      ),
                    ),
```

```
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

This code implements a "LinkScreen" widget that functions as a screen for handling user interactions pertaining to video meetings and chat features in the application. The application incorporates Firebase for user authentication and Firestore for data retrieval. The page displays user information retrieved from Firestore and offers the ability to start a new meeting with a created ID and begin calls, or join an existing meeting by inputting a meeting link. It dynamically shows and hides text areas in response to user activities. In addition, it provides a logout function and integrates user interface components with a gradient backdrop and many buttons for various operations, such as making phone calls and logging out.

Code for "signup_screen.dart":

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
import '../linkscreen.dart';
import '../reusable_widgets/reusable_widget.dart';
import '../utils/color_utils.dart';
import 'home_screen.dart';

Future<void> addDataToFirestore(String collectionName, Map<String,
dynamic> data) async {
  try {
    await
FirebaseFirestore.instance.collection(collectionName).add(data);
    print('Data added to Firestore successfully!');
  } catch (e) {
```

```
    print('Error adding data to Firestore: $e');
  }
}

class SignUpScreen extends StatefulWidget {
  const SignUpScreen({Key? key}) : super(key: key);

  @override
  _SignUpScreenState createState() => _SignUpScreenState();
}

class _SignUpScreenState extends State<SignUpScreen> {
  TextEditingController _passwordTextController =
TextEditingController();
  TextEditingController _emailTextController =
TextEditingController();
  TextEditingController _userNameTextController =
TextEditingController();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      extendBodyBehindAppBar: true,
      appBar: AppBar(
        backgroundColor: Colors.transparent,
        elevation: 0,
        title: const Text(
          "Sign Up",
          style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold),
        ),
      ),
      body: Container(
          width: MediaQuery.of(context).size.width,
          height: MediaQuery.of(context).size.height,
          decoration: BoxDecoration(
              gradient: LinearGradient(colors: [
```

```
                hexStringToColor("CB2B93"),
                hexStringToColor("9546C4"),
                hexStringToColor("5E61F4")
              ], begin: Alignment.topCenter, end:
Alignment.bottomCenter)),
          child: SingleChildScrollView(
              child: Padding(
                padding: EdgeInsets.fromLTRB(20, 120, 20, 0),
                child: Column(
                  children: <Widget>[
                    const SizedBox(
                      height: 20,
                    ),
                    reusableTextField("Enter UserName",
Icons.person_outline, false,
                        _userNameTextController),
                    const SizedBox(
                      height: 20,
                    ),
                    reusableTextField("Enter Email Id",
Icons.person_outline, false,
                        _emailTextController),
                    const SizedBox(
                      height: 20,
                    ),
                    reusableTextField("Enter Password",
Icons.lock_outlined, true,
                        _passwordTextController),
                    const SizedBox(
                      height: 20,
                    ),
                    firebaseUIButton(context, "Sign Up", () async {

                      Map<String, dynamic> userData = {
                        'name': _userNameTextController.text,
                        'email': _emailTextController.text,
                        'age': 25,
```

```
                        };

                        await addDataToFirestore('users', userData);

                        FirebaseAuth.instance
                            .createUserWithEmailAndPassword(
                            email: _emailTextController.text,
                            password: _passwordTextController.text)
                            .then((value) {
                          print("Created New Account");
                          Navigator.push(context,
                              MaterialPageRoute(builder: (context) =>
LinkScreen()));
                        }).onError((error, stackTrace) {
                          print("Error ${error.toString()}");
                          String errorMessage = "${error.toString()}";
                          int startIndex = errorMessage.indexOf('] ')
+ 2;

                          String extractedMessage =
errorMessage.substring(startIndex);

                          Fluttertoast.showToast(
                            msg: extractedMessage,
                            toastLength: Toast.LENGTH_SHORT,
                            gravity: ToastGravity.BOTTOM,
                            backgroundColor: Colors.grey,
                            textColor: Colors.white,
                            fontSize: 16.0,
                          );
                        });
                      })
                  ],
                ),
              ))),
    );
```

```
  }
}
```

The SignUpScreen widget, which is in charge of user registration within the application, is defined by the provided Flutter code. It creates an interface with text fields where users can input their email address, password, and username. The screen has a gradient background, user data entry forms, and a "Sign Up" button that uploads user information to Firestore and initiates user registration with Firebase Authentication. After the account creation process is finished, the user is directed to the LinkScreen where they can use chat and meeting features. If the registration process fails, the user is notified of any problems found throughout the signup process by the display of a toast message containing the specific fault.

Code for "reset_password.dart":

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import '../reusable_widgets/reusable_widget.dart';
import '../utils/color_utils.dart';

class ResetPassword extends StatefulWidget {
  const ResetPassword({Key? key}) : super(key: key);

  @override
  _ResetPasswordState createState() => _ResetPasswordState();
}

class _ResetPasswordState extends State<ResetPassword> {
  TextEditingController _emailTextController =
TextEditingController();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      extendBodyBehindAppBar: true,
      appBar: AppBar(
        backgroundColor: Colors.transparent,
        elevation: 0,
        title: const Text(
          "Reset Password",
```

```
          style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold),
        ),
      ),
      body: Container(
          width: MediaQuery.of(context).size.width,
          height: MediaQuery.of(context).size.height,
          decoration: BoxDecoration(
              gradient: LinearGradient(colors: [
            hexStringToColor("CB2B93"),
            hexStringToColor("9546C4"),
            hexStringToColor("5E61F4")
          ], begin: Alignment.topCenter, end:
Alignment.bottomCenter)),
          child: SingleChildScrollView(
              child: Padding(
            padding: EdgeInsets.fromLTRB(20, 120, 20, 0),
            child: Column(
              children: <Widget>[
                const SizedBox(
                  height: 20,
                ),
                reusableTextField("Enter Email Id",
Icons.person_outline, false,
                    _emailTextController),
                const SizedBox(
                  height: 20,
                ),
                firebaseUIButton(context, "Reset Password", () {
                  FirebaseAuth.instance
                      .sendPasswordResetEmail(email:
_emailTextController.text)
                      .then((value) => Navigator.of(context).pop());
                })
              ],
            ),
          ))),
```

```
    );
  }
}
```

The "ResetPassword" class, defined in reset_password, is in charge of changing a user's password inside the programme. It generates an interface with a "Reset Password" button and a text field where the user may enter their email address. An email address entry field is part of the user interface design. The widget sends a password reset email to the provided email address when the "Reset Password" button is clicked, starting a password reset procedure via Firebase Authentication. The screen that shows that the password reset email has been received is dismissed when the reset request has been submitted successfully.

Code for "utils.dart":

```
class Utils{
  static const int chatappId = 1261518879;
  static const String chatappSign =

"76a8d22d0d2faff6f24c5a8c88ce8b74e3b2653e7ad704463bc7005ab313dbe3";
}
```

The code that is provided defines a class called "Utils," which holds static constants associated with a chat programme. In particular, it contains the string constant chatappSign, whose value is a long hexadecimal character, and the integer constant chatappId, which has the value 1261518879. These constants stand for credentials for identification and authentication, like an application ID and matching application sign or token used in a chat programme.

Code for "homescreen.dart":

```
import 'package:TeleCon/screens/signin_screen.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';


class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  _HomeScreenState createState() => _HomeScreenState();
```

```
}

class _HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: ElevatedButton(
          child: Text("Logout"),
          onPressed: () {
            FirebaseAuth.instance.signOut().then((value) {
              print("Signed Out");
              Navigator.push(context,
                  MaterialPageRoute(builder: (context) =>
SignInScreen()));
            });
          },
        ),
      ),
    );
  }
}
```

The application's HomeScreen widget, which is created using the supplied Flutter code, is in charge of showing a basic user interface. It is comprised of a single raised button that is labelled "Logout." Pressing the button ends the current user session by initiating a sign-out activity via Firebase Authentication. After a successful sign-out, the user is able to log back in by returning to the SignInScreen and printing "Signed Out" to the console. This screen mainly acts as the application's logout mechanism, sending users back to the sign-in screen after they log out.

Code for "CallUI.dart":

```
import 'package:TeleCon/utils/color_utils.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
```

```dart
import 'package:zego_zimkit/zego_zimkit.dart';

import 'call.dart';

class ChatUI extends StatelessWidget {
  ChatUI({Key? key,required this.email,required this.username}) :
super(key: key);

  User? user = FirebaseAuth.instance.currentUser;
  var docid;
  var username;
  var email;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(email,style: TextStyle(fontSize: 18),),
        backgroundColor: hexStringToColor("CB2B93"),
        actions: <Widget>[
          IconButton(
            onPressed: () async {

              ZIMKit().showDefaultNewPeerChatDialog(context);
            }, icon: Icon(Icons.send),

          ),
        ],
      ),
      body:
    Container(
    width: MediaQuery.of(context).size.width,
    height: MediaQuery.of(context).size.height,
    decoration: BoxDecoration(
    gradient: LinearGradient(colors: [
    hexStringToColor("CB2B93"),
    hexStringToColor("9546C4"),
```

```
    hexStringToColor("5E61F4")
    ], begin: Alignment.topCenter, end: Alignment.bottomCenter)),
      child:  ZIMKitConversationListView(
        onPressed: (context, conversation, defaultAction) {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) {
                return ZIMKitMessageListPage(
                  conversationID: conversation.id,
                  conversationType: conversation.type,
                );
              },
            ),
          );
        },
      ),
    ),
   );
  }
}
```

The "ChatUI" class in CallUI.dart is defined by the code and is in charge of presenting a chat interface inside the application. It generates a scaffold with an AppBar that shows the user's email as the title and an action button (IconButton) with a send icon as its representation. A "ZIMKitConversationListView" is contained in a container with a gradient backdrop that is part of the scaffold's body. This widget called "ZIMKit" makes it easier to see a list of discussions. When a discussion item in the list is clicked, the user is taken to the "ZIMKitMessageListPage," which most likely displays the messages in that particular conversation. Users may efficiently engage with their chat conversations and messages thanks to the chat interface's integration of the Zego ZIMKit SDK for managing and displaying conversations and messages.

4.2 Firebase Integration

After going to “console.firebase.google.com”, we need to create a new project and give it a proper name. Then from Build menu, Authentication is chosen and “Email/Password” was chosen as Sign-in provider. It is shown in figure 5.1 below.

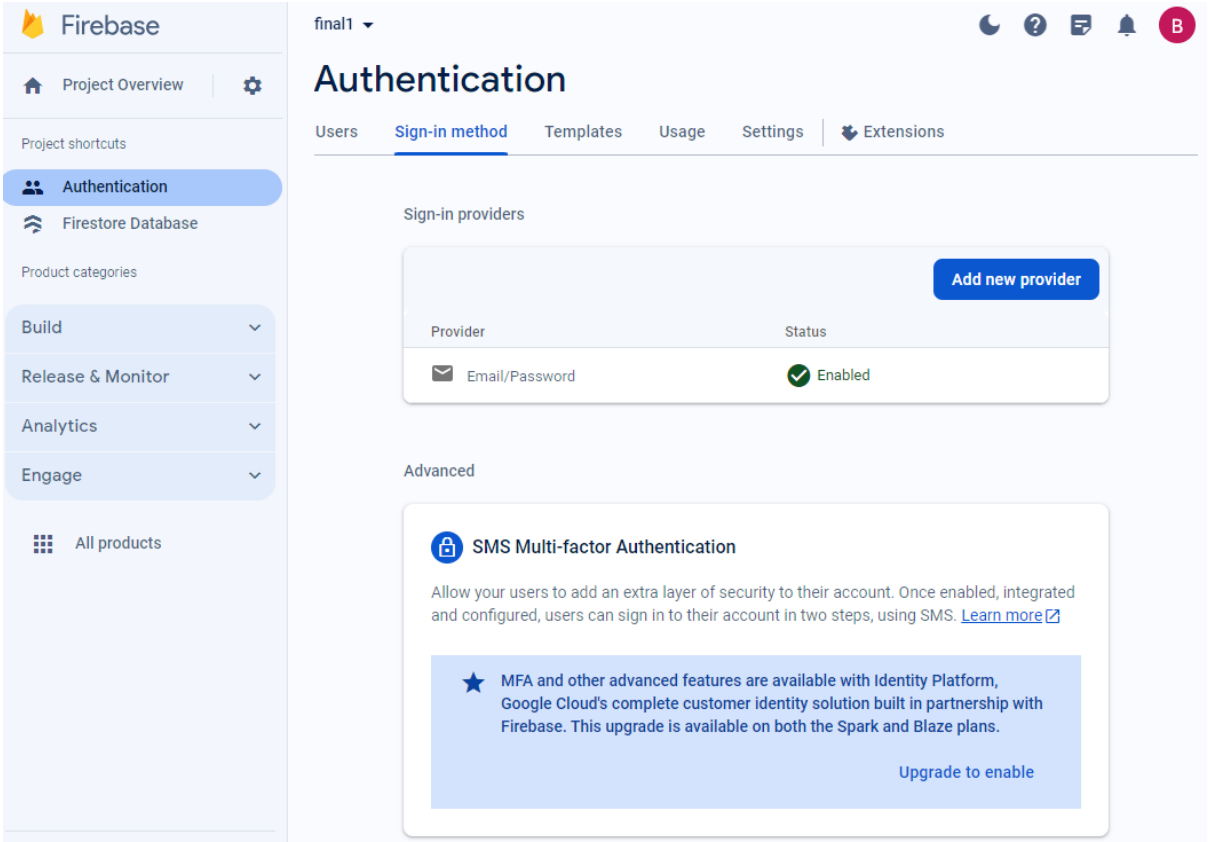


Figure 4.1 “Email/Password” as Sign-in method

To connect it with the android app, we need to perform the following steps:

1. Register the app by entering “Android Package name”, “App nickname”, “Debug signing certificate SHA-1”

2. Switch to the Project view in Android Studio to see the project root directory. Download “google-services.json” file and move it into the module (app-level) root directory.

3. To make the “google-services.json” config values accessible to Firebase SDKs, the Google services Gradle plugin is needed. the plugin was added as a dependency to project-level build.gradle.kts file:

Root-level (project-level) Gradle file (<project>/build.gradle.kts):

```
plugins {
  // ...
  // Add the dependency for the Google services Gradle plugin
  id("com.google.gms.google-services") version "4.4.0" apply false
}
```

4. Then, in the module (app-level) build.gradle.kts file, both the google-services plugin and any Firebase SDKs were added that I want to use in your app:

```
plugins {


    id("com.android.application")
    // Add the Google services Gradle plugin
    id("com.google.gms.google-services")
    ...
}


dependencies {
     // Import the Firebase BoM
     implementation(platform("com.google.firebase:firebase-bom:32.6.0"))

     // TODO: Add the dependencies for Firebase products you want to use
     // When using the BoM, don't specify versions in Firebase dependencies
      implementation("com.google.firebase:firebase-analytics")

     // Add the dependencies for any other desired Firebase products
     // https://firebase.google.com/docs/android/setup#available-libraries
}
```

The firebase stored the data for sign-in credentials in the Authentication/Users tab which is shown in figure 5.2. Personal data from sign-up form are stored in firestore database and illustrated in figure 5.3.

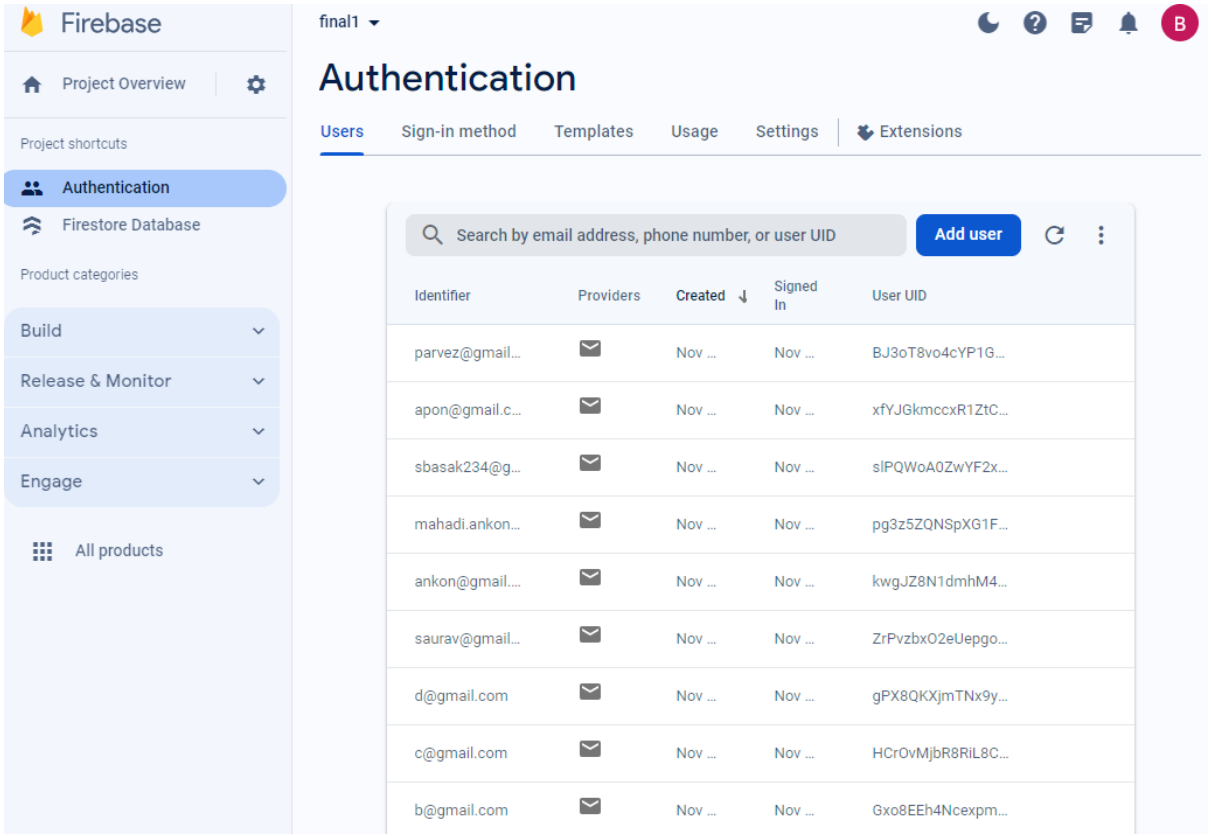


Figure 4.2 Sign-in credentials are stored in firebase

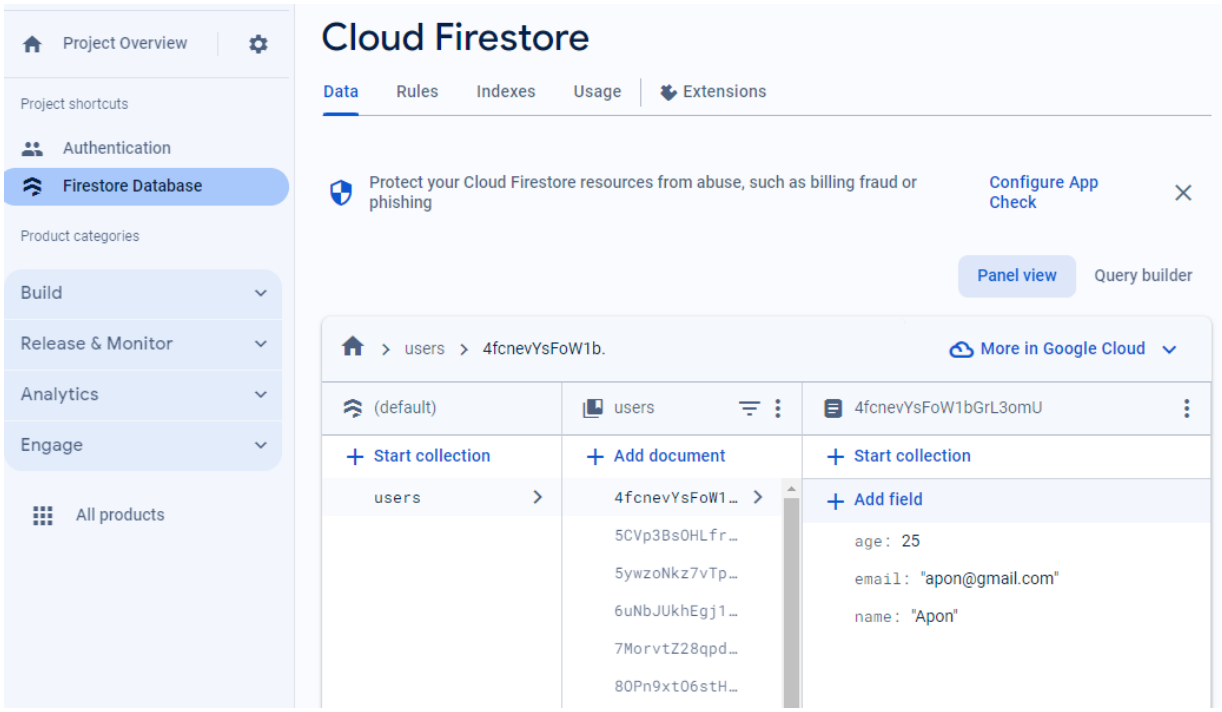


Figure 4.3 Personal data from sign-up form are stored in Firestore Database

4.3 ZEGOCLOUD Integration

To integrate the SDK, the following steps were performed:

1. Adding ZegoUIKitPrebuiltCall as dependencies:

The following code was run in the project root directory.

```
flutter pub add zego_uikit_prebuilt_call
```

2. Importing the SDK:

Now in the Dart code, the prebuilt Call Kit SDK was imported.

```
import 'package:zego_uikit_prebuilt_call/zego_uikit_prebuilt_call.dart';
```

3. Using the ZegoUIKitPrebuiltCall in the project:

Went to ZEGOCLOUD Admin Console, got the appID and appSign of my project. Specified the userID and userName for connecting the Call Kit service. Created a callID that represents the call I want to make.

```
class CallPage extends StatelessWidget {
 const CallPage({Key? key, required this.callID}) : super(key: key);
 final String callID;

 @override
 Widget build(BuildContext context) {
  return ZegoUIKitPrebuiltCall(
    appID: yourAppID, // Fill in the appID that you get from ZEGOCLOUD Admin Console.
    appSign: yourAppSign, // Fill in the appSign that you get from ZEGOCLOUD Admin Console.
    userID: 'user_id',
    userName: 'user_name',
    callID: callID,
    // You can also use groupVideo/groupVoice/oneOnOneVoice to make more types of calls.
    config: ZegoUIKitPrebuiltCallConfig.oneOnOneVideoCall()
```

```
      ..onOnlySelfInRoom = () => Navigator.of(context).pop(),
    );
  }
}
```

4. Configure the project:

For Android: Modified the compileSdkVersion to 33 in the project/android/app/build.gradle file.

5. Add app permissions:

Opened the file untitled2/app/src/main/AndroidManifest.xml, and added the following code:

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
<uses-permission android:name="android.permission.WAKE_LOCK" />
```

6. Run & Test:

The Run or Debug to run was clicked and the App was tested on several devices.

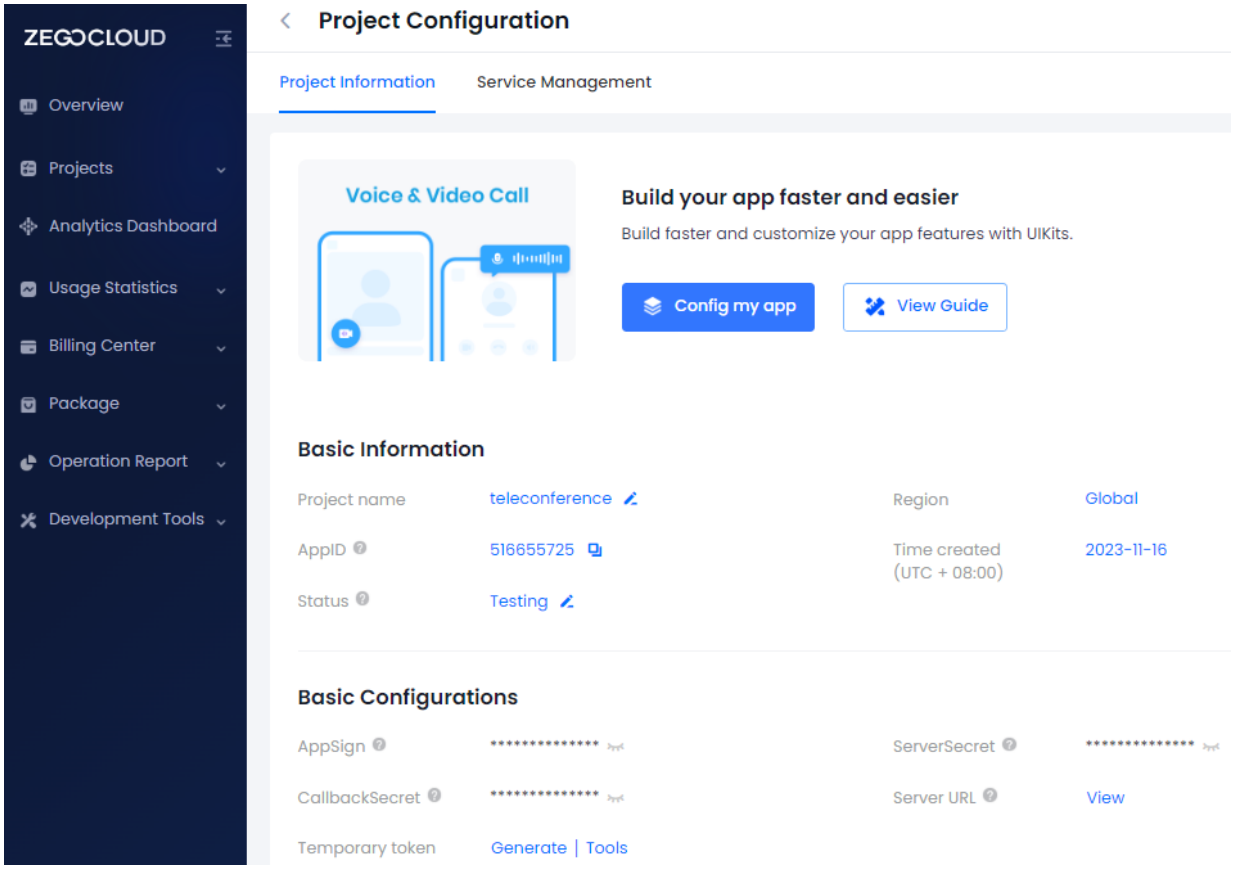


Figure 4.4 ZEGOCLOUD Project Configuration for Voice & Video Call

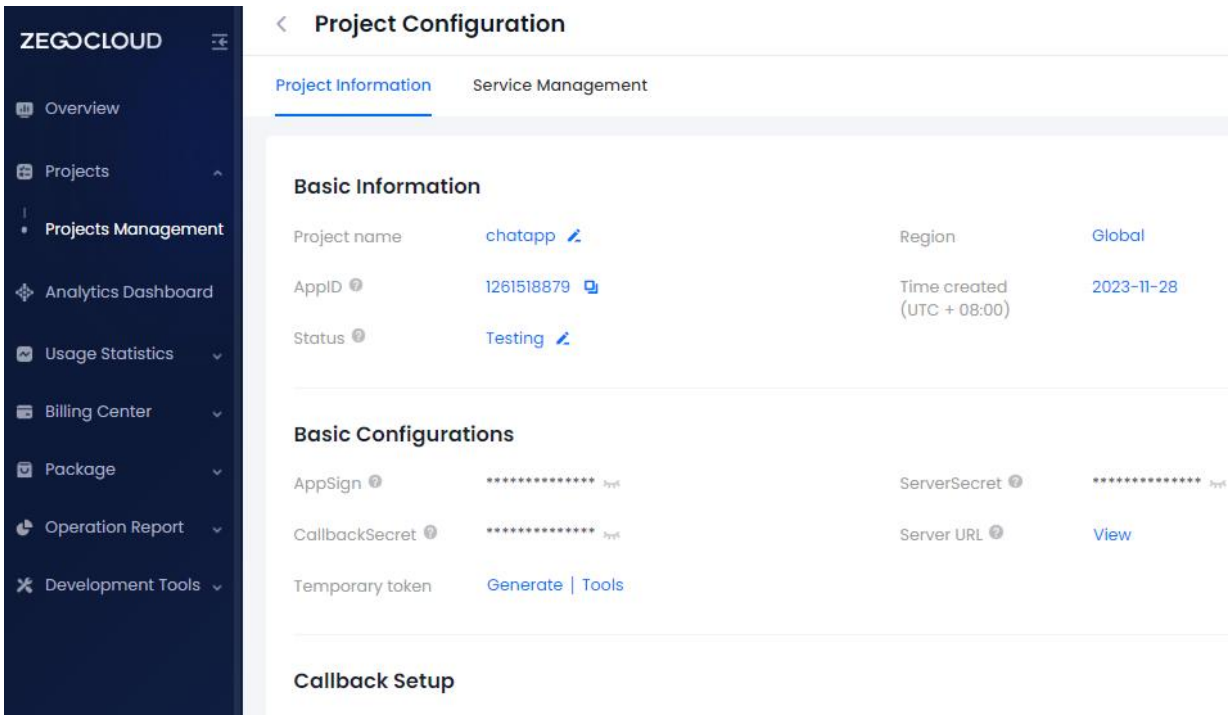


Figure 4.5 ZEGOCLOUD Project Configuration for Chat App

4.4 App Link Generation

The apk file was build and uploaded to google drive. A google drive link was used to generate a QR-Code of the apk file.

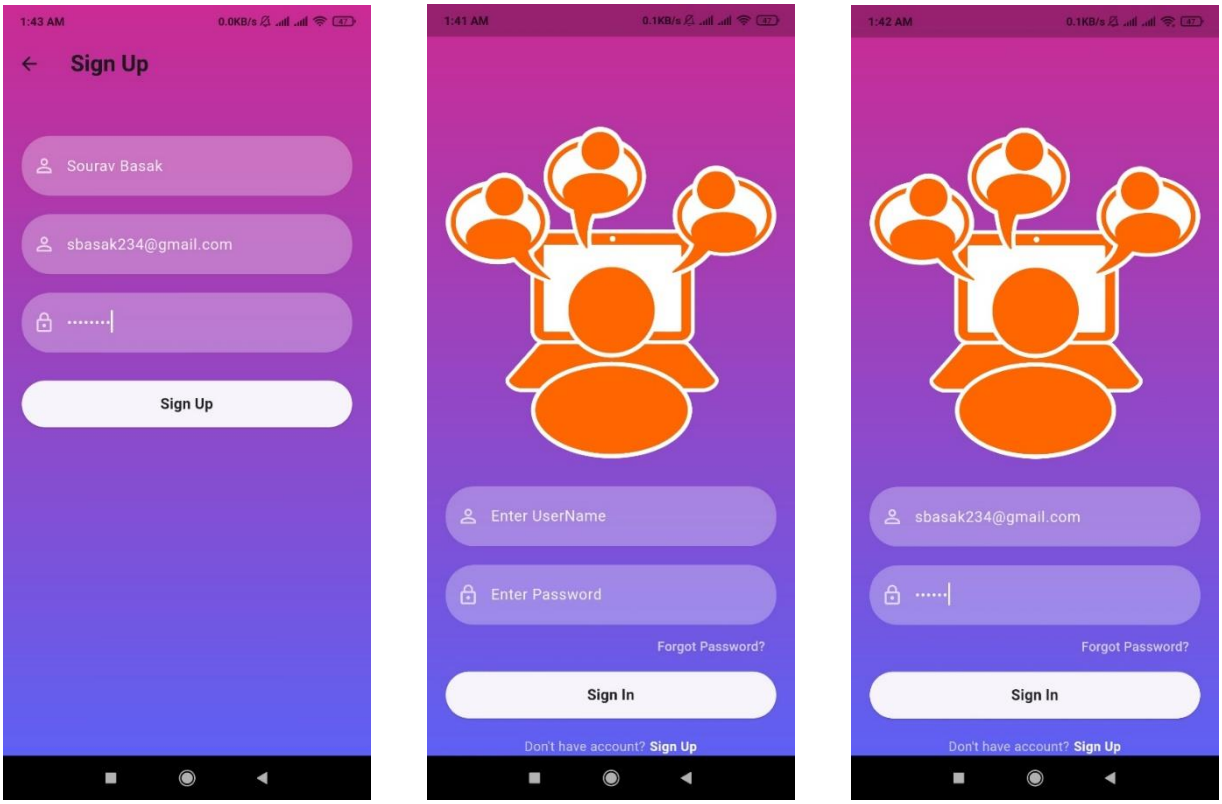
Figure 4.6 Generated QR-Code of the App

Chapter 5

Results and Discussion

5.1 Results

Here, the sign-up page is shown in figure 5.1 (a), sign-in page in shown in figure 5.1 (b).



(a) (b) (c)

Figure 5.1 Pages for creating an ID and using that ID to enter the main interface

(a) Sign Up page, (b) Sign In page, (b) Sign In page with credential

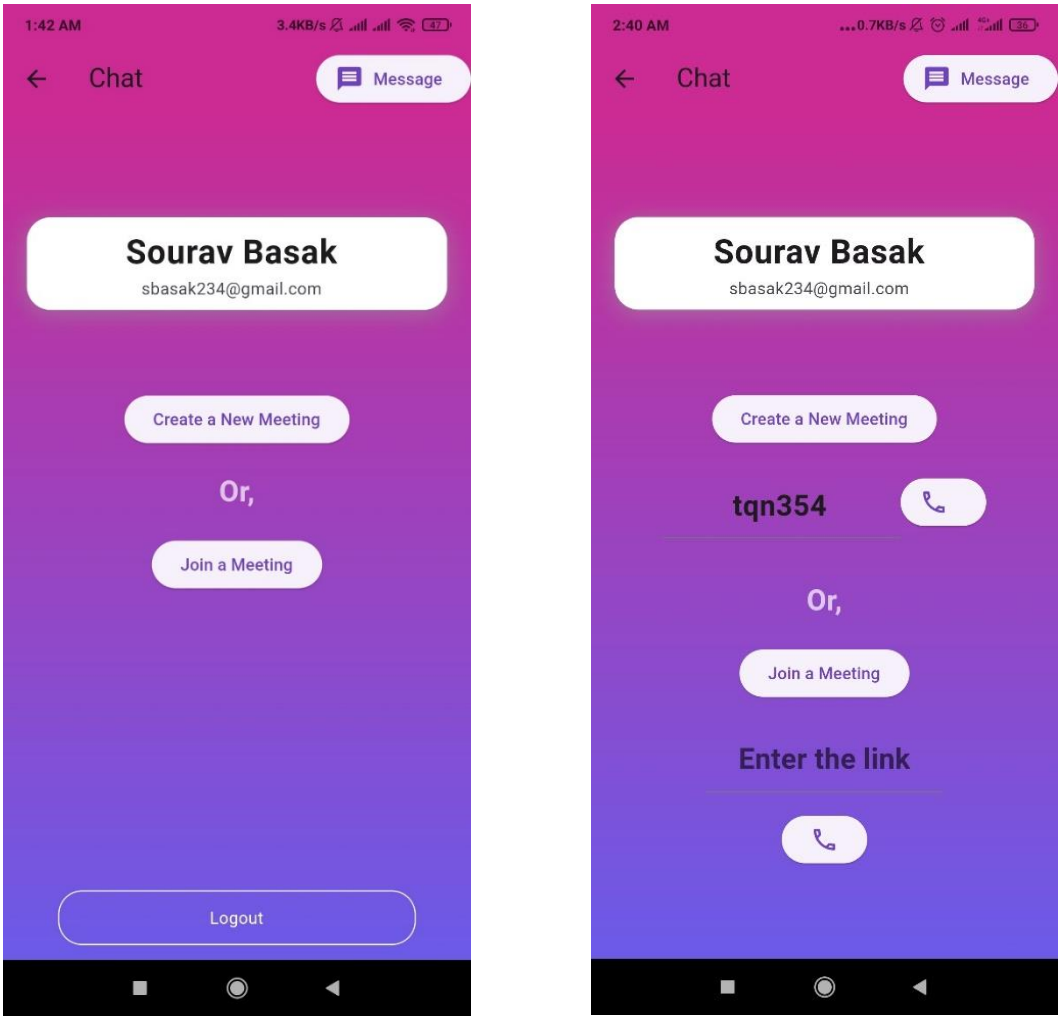
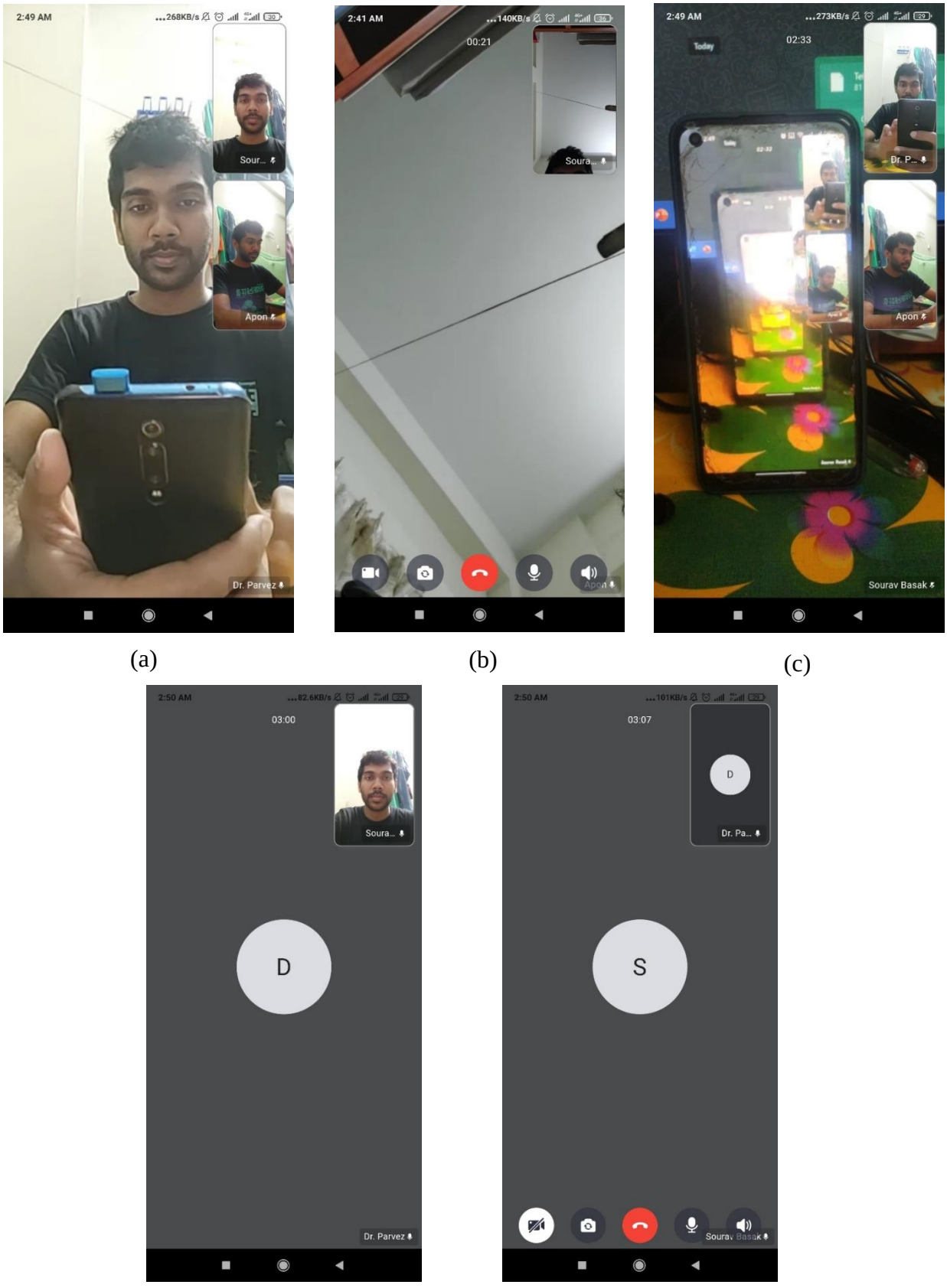


Figure 5.2 Teleconferencing Interface for creating a new meeting and joining a meeting



(a) (b) (c)

(d) (e)

Figure 5.3 Teleconferencing (a) Meeting among 3 persons, (b) Meeting among 2 persons, (c) Function of the back camera, (d) Turing off the camera and (e) Audio call Function

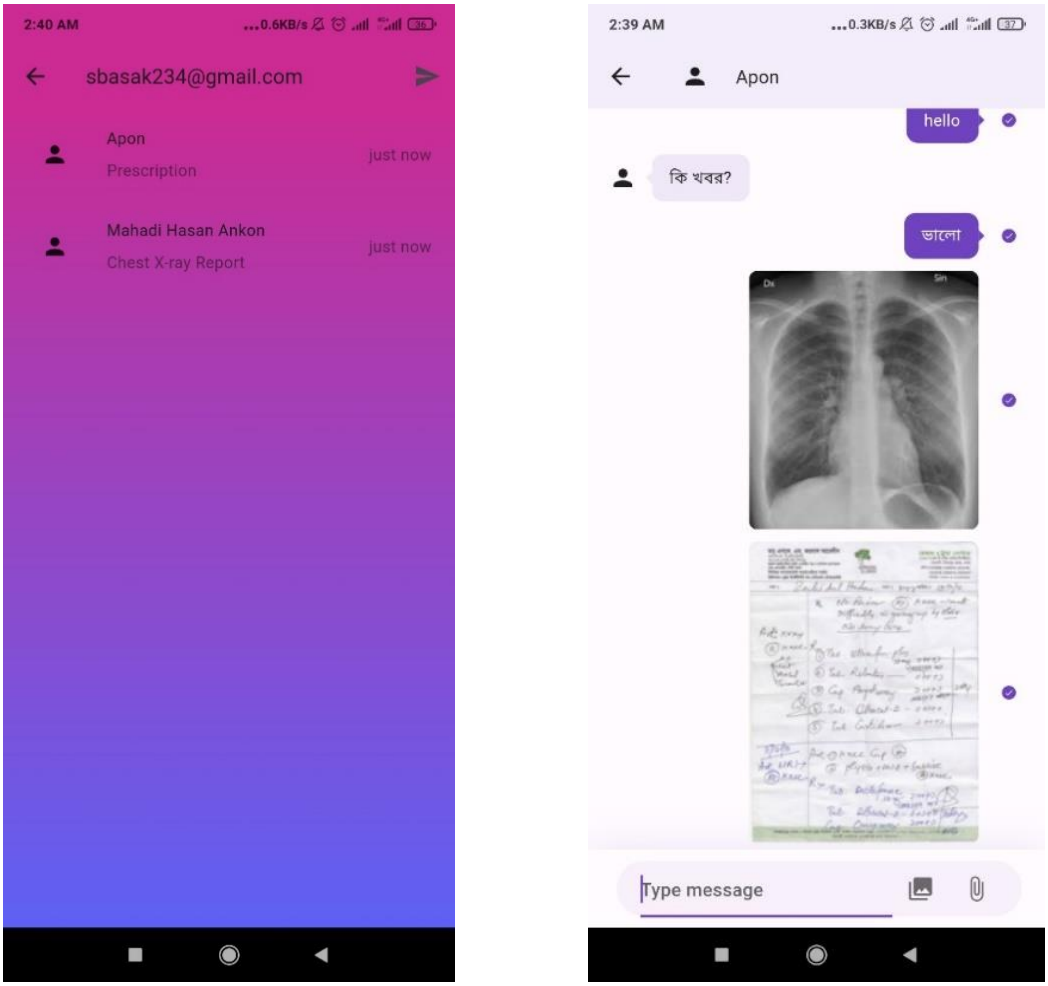


Figure 5.4 Chatbox Interface (a) UI for Chatbox, (b) Sending medical image, reports and prescription

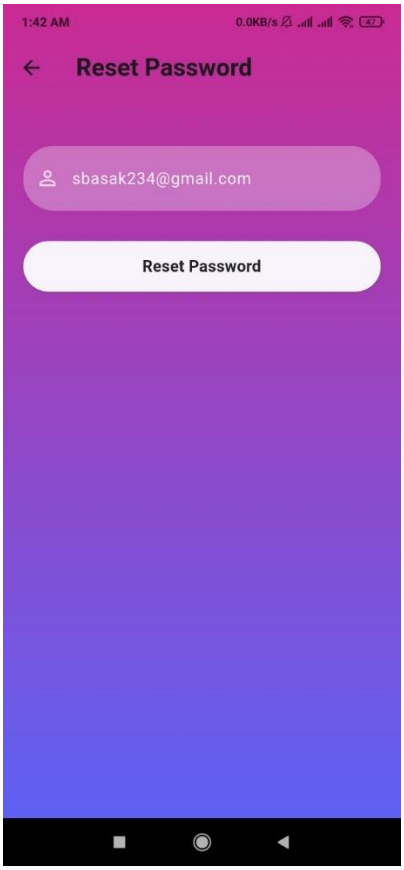


Figure 5.5 Reset Password Page

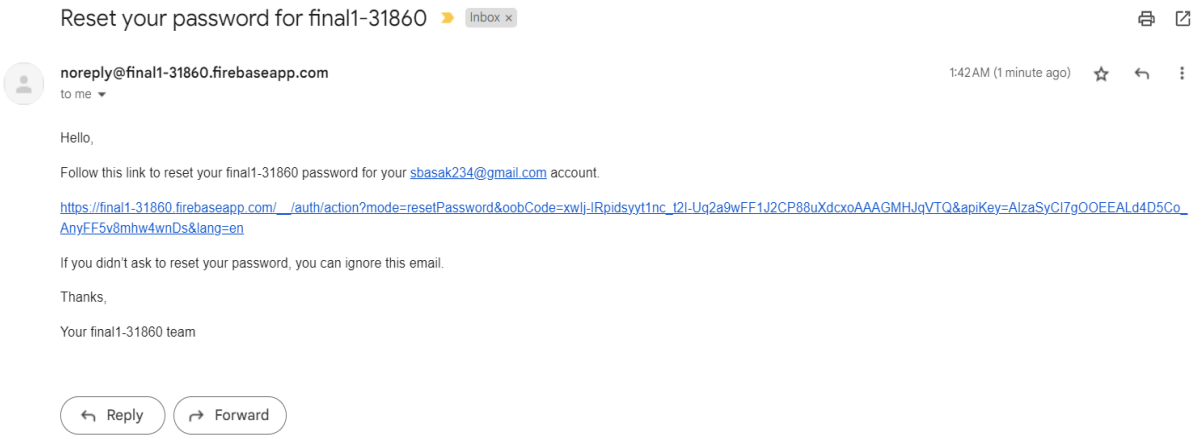


Figure 5.6 Reset Password E-mail and link for resetting

5.2 Discussion

This project centers on the development of a cutting-edge Video Conferencing App tailored specifically for telemedicine and healthcare services. The App was build using Android studio, java language, Firebase and ZEGOCLOUD. The codes were written in java language and Android Studio was used as Integrated Development Environment (IDE) for the purpose of Android application development. The pages for creating an ID and using that ID to enter the main interface was shown in figure 5.1. Sign-up and sign-in page was shown in figure 5.1 (a) & 5.1 (b) respectively. After Signing in, the user enters the interface for video conferencing and messaging. Here, one can create a meeting, join a meeting or message with people. This interface is shown in figure 5.2. A meeting among 3 persons and a call between 2 people are shown in figure 5.3 (a) and (b) respectively. The video call was connected without delaying and the data transfer was quite fast, So, there was no lagging or delay. The sound quality was quite good and the interface worked smoothly. Back camera function, camera off function and audio call function was depicted in figure 5.3 (c), (d) and (e). The chat box interface was shown in figure 5.4. Medical images, reports and prescription was sent via message. This function also worked properly. It also supported many languages, such as, Bangla, English or Hindi. The reset password page was also checked. It also worked perfectly. Reset password page UI is shown in figure 5.5. Reset Password E-mail and link for resetting was shown in figure 5.6. Thus, the main objectives of the project were fulfilled

5.3 Limitations

Although the main objectives of the project are fulfilled, but still there are some limitations and room for improvements. Only Email/Password Authentication is used in this app, here other accounts options, such as, “Log in using Facebook”, “Log in using Google” and others can be integrated. The project can be made more robust by designing the API without using a pre-built API. More personal information section for both doctor and patient could be added. The video calling can be made smoother and more robust. The email verification wasn’t added while signing up, so any email can be used to signup in the app. These limitations should be a matter of attention for future updates.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The main goals of the project were effectively fulfilled by developing this video conferencing software using Android Studio, Java, and integration with Firebase and Zegocloud. This allowed for the creation of video calls and messaging for telemedicine. The app accomplishes these objectives and provides a solid basis for future improvements. Subsequent versions may have enhanced security features, such as login and sign-in choices, to raise the app's security bar. Furthermore, the integration of medical devices for health parameter monitoring is a viable option for enhancing the effectiveness and range of telehealth services, indicating an interesting path for further advancement and improvement.

6.2 Future Work

It is possible to develop this video conferencing app further. To reduce the limitations of this project, the options below can be considered.

- Security options can be further enhanced.
- More screens can be created to add different types of telehealth options.
- More log in feature and Email ID Authentication using Firebase can be added for having better security.
- Features such as smooth device switching during meetings or noise reduction should be a matter of attention.

References:

[1] NEJM Catalyst, "What Is Telehealth?," *Catalyst Carryover*, vol. 4, no. 1, Feb. 2018, doi: 10.1056/CAT.18.0268.

[2] "Teleconference," *Wikipedia*. Sep. 03, 2023. Accessed: Nov. 30, 2023. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Teleconference&oldid=1173591396

[3] "What is Video Conferencing?," Unified Communications. Accessed: Nov. 30, 2023. [Online]. Available: https://www.techtarget.com/searchunifiedcommunications/definition/video-conference

[4] "Video Conferencing: How It Works, How to Use It, Top Platforms," Investopedia. Accessed: Nov. 30, 2023. [Online]. Available: https://www.investopedia.com/terms/v/video-conferencing.asp

[5] "What is video conferencing? — Definition by TrueConf," TrueConf. Accessed: Nov. 30, 2023. [Online]. Available: https://trueconf.com/what-is-video-conferencing.html

[6] "7 Advantages of Video Conferencing in Telemedicine." Accessed: Nov. 30, 2023. [Online]. Available: https://www.megameeting.com/news/advantages-of-video-conferencing-with-patients/

[7] "Meet Android Studio," Android Developers. Accessed: Nov. 30, 2023. [Online]. Available: https://developer.android.com/studio/intro

[8] "Firebase Realtime Database," Firebase. Accessed: Nov. 30, 2023. [Online]. Available: https://firebase.google.com/docs/database

[9] "Kuk Jiang | Co-founder - ZEGOCLOUD PTE. LTD," Forbes Technology Council. Accessed: Nov. 30, 2023. [Online]. Available: https://councils.forbes.com/profile/Kuk-Jiang-Co-founder-ZEGOCLOUD-PTE-LTD/8784c6bd-2e6b-4eef-bebf-a23454a3f813